

计算机系统基础 作业三

学号：2410668

姓名：吕昊远

平时作业三

- 一、为什么用时钟边缘（下降沿）做控制信号？
- 二、`_gmon_start_` 的含义



- 三、为什么ori指令是0扩展，而load是符号扩展

一、为什么用时钟边缘下降沿做控制信号？

时序逻辑电路的状态必须在一个确定的瞬间时刻才发生改变，否则就会出现竞争冒险和空翻的问题。如果用电平去做控制信号，那么在这段电平有效的时间里，寄存器会一直处于打开状态，输入端只要有变化就会被传进去，于是可能在同一个时钟周期内被反复改写，也就是空翻。而边沿只是时间轴上的一个点，它给出一个明确的触发瞬间，能保证每个寄存器在一个时钟周期内只被更新一次，时序关系清晰可靠。

之所以选下降沿这一侧，可以从一个时钟周期内的信号变化来看。在时钟有效的那段时间里，数据沿着组合逻辑向前传播并逐渐稳定下来，等到下降沿这一刻，输入信号已经稳定，再由它把结果存入寄存器，输出端才反映新值。这期间输入的变化不会影响输出，有效地避开了竞争冒险，保证了数据的可靠传送。

二、`__gmon_start__`的含义

`__gmon_start__`是与 GNU 性能剖析相关的一个函数符号，即 gprof monitor。其作用是初始化程序的性能监测机制。它被引用在 C 运行时的启动代码 crt 启动部分里，是一个弱符号。

当程序用 `-pg` 选项编译、开启了 gprof 剖析支持时，这个符号会被真正定义，启动时调用它来初始化剖析所需的数据结构，从而让 `mcount` 在每次函数调用时被记录，程序结束后生成 `gmon.out` 供 gprof 分析；而当没有开启剖析时，这个弱符号解析为空，启动代码会先判断它是否非空再决定是否调用，所以这种情况下这次调用实际上是被跳过的。

至于反汇编里看到的 `<__gmon_start__@plt+16>`，这里 `@plt` 表示这个符号是通过 PLT（Procedure Linkage Table，过程链接表）来访问的。PLT 是动态链接中用来对共享库函数做地址解析的一张跳转表，程序第一次调用时才把符号绑定到真正的函数地址上，之后再调用就直接跳转。

三、为什么 `ori` 指令是 0 扩展，而 `load` 是符号扩展？

两者的差别主要在于指令里那个 16 位立即数代表什么，指令中的立即数只有 16 位，而运算要在 32 位上进行，所以必须先扩展成 32 位，因此两者的区

别其实在于高 16 位补什么。

对 ori 这类逻辑指令来说，立即数被当作一个逻辑数，是按位参与逻辑运算的，本身没有正负和符号的概念。逻辑指令的设计意图是只对寄存器的低 16 位做操作、不影响高 16 位，所以扩展时高 16 位必须补 0，这样高 16 位与 0 相或后才能保持原值不变。如果改用符号扩展，一旦立即数最高位是 1，高 16 位就会被填成全 1，或运算之后寄存器的高半部分全被强行置为 1，显然不对。因此逻辑立即数必须要用 0 扩展。

而 load 里的立即数则是一个地址偏移量，要和基址寄存器相加来算出访存地址。这个偏移量是可以为负的，表示朝低地址方向偏移。它是用补码表示的有符号数，要在从 16 位扩展到 32 位时保持数值与符号不变，就必须做符号扩展：负数前补 1 仍是负数，正数前补 0 仍是正数，算出来的地址才正确；如果对一个负偏移量也做 0 扩展，就会变成一个很大的正数，导致地址彻底算错。

总而言之，两者的差别就在于立即数被当作什么来用，逻辑运算把它当作无符号的位模式，所以采用 0 扩展；访存与算术把它当作有符号的数值偏移，所以采用符号扩展。